CLIFFGUARD

An Edge-Deployable Defense System Against Prompt Injection Attacks in Quantized Language Models

Engineering Project Proposal

Submitted by: Trilochan Bhandari
Department of Computer Science and Engineering
Kathmandu University, Dhulikhel, Nepal
trilochan2625@student.ku.edu.np

July 2026

Abstract

Large Language Models (LLMs) are increasingly deployed on small, low-power edge devices using quantization — a compression technique that shrinks model size but can also weaken built-in safety behaviour, making models more vulnerable to prompt injection. Existing defense tools target full-precision, server-side models and ignore this quantization-related weakening, nor are they optimised for constrained hardware. CLIFFGUARD is a lightweight, layered defense system built specifically for quantized, edge-deployed LLMs. It combines fast input filtering, internal and black-box behaviour monitoring, real-time output monitoring, an adaptive checking strategy, and a hardware-aware router that adjusts which checks run based on available memory and processing power. Detection thresholds are calibrated separately per quantization level so accuracy is preserved as the model is compressed. The system will be implemented and tested on Llama-3.2-3B-Instruct across multiple quantization levels and demonstrated on a Raspberry Pi 5. The expected outcome is a working, tested prototype offering a practical, low-cost way to protect compressed LLMs on edge devices, with a clear path for future extension.

1. Introduction

LLMs such as Llama and Qwen are increasingly deployed not only on cloud servers but on small, low-power "edge" devices — Raspberry Pi boards, smartphones, and embedded chips. Quantization reduces the precision of a model's numbers (e.g., 16-bit to 4-bit) so it fits and runs faster on such devices. CLIFFGUARD is an engineering project that designs and implements a lightweight security system to protect quantized, edge-deployed LLMs from prompt injection — attacks where a malicious user tricks the model into ignoring its safety instructions. The project focuses on building working software components, testing them on real models and hardware, and demonstrating deployment on resource-constrained devices.

2. Background

LLMs are trained to refuse harmful requests via Reinforcement Learning from Human Feedback (RLHF), giving them an internal "safety sense." However, quantization for edge deployment can weaken this behaviour: published studies show a model that safely refuses a harmful request at full precision (FP16) may comply once quantized to lower precision (4-bit / 3-bit GGUF). This project calls this weakening the "safety cliff." Meanwhile, prompt injection remains one of the most common practical attacks against LLM-based applications, with attackers hiding instructions inside user input, documents, or web pages. Existing tools — Llama Guard, NeMo Guardrails, PromptGuard — are designed for full-sized server-side models; very few work efficiently on edge devices, and none specifically account for the added risk from quantization.

3. Problem Statement

No practical, lightweight defense system currently:

- Works efficiently on small edge devices with limited memory and no GPU (e.g., Raspberry Pi).
- Accounts for the fact that a model's safety behaviour can change after quantization.
- Can adapt its detection strategy over time instead of using one fixed rule.
- Provides a fallback for closed, "black-box" models whose internals cannot be observed.

This leaves a real security gap: developers deploying compressed LLMs on edge devices for assistants, chatbots, or IoT applications have no reliable, low-cost way to detect quantization-degraded safety or to block prompt injection in real time. CLIFFGUARD is proposed to close this gap through a working, engineered software system.

4. Objectives

General objective: design, build, and test a multi-layer defense system that protects quantized LLMs on edge hardware from prompt injection attacks. Specific objectives:

1. Design a layered architecture: input filtering, internal-state monitoring, output monitoring, and adaptive decision-making.
2. Implement calibration that adjusts detection thresholds separately per quantization level (e.g., FP16 and 4-bit NF4) to keep accuracy consistent across compression levels.
3. Implement a lightweight streaming monitor that flags suspicious output patterns during generation, without buffering the full output.
4. Implement a hardware-aware routing mechanism that selects which security checks run based on available device memory and compute.
5. Test the complete system on Llama-3.2-3B-Instruct at two or more quantization levels, measuring detection accuracy and speed.
6. Deploy and demonstrate the system on a real edge device, such as a Raspberry Pi 5.

5. Scope

This is a final working prototype rather than an exhaustive research study. In scope: implementing all core CLIFFGUARD modules as working code; testing on Llama-3.2-3B-Instruct across FP16, NF4, and (if time permits) a GGUF 4-bit format; demonstrating deployment on at least one edge-device tier (e.g., Raspberry Pi 5) and, if resources allow, a lower-memory device; evaluating using public safety-testing datasets (e.g., AdvBench).

Out of scope, given semester time/resource limits: formal mathematical security proofs; large-scale evaluation across many model families and quantization schemes; training a fully autonomous RL policy from live traffic (a simplified rule-assisted adaptive mechanism is used instead); and building a certified, production-grade security product — the outcome is a prototype and demonstration.

6. Proposed System Design

CLIFFGUARD is a pipeline of six practical modules surrounding the protected LLM:

6.1 Input Gate Module (VESTIBULE)

Checks incoming prompts before they reach the model: measures text compressibility (attack text often compresses differently from normal language), tags text from external sources (documents/websites) so the model can distinguish it from the user's own instructions, and scans for unusual character patterns such as hidden encodings or excessive symbols.

6.2 Internal Observer Module (PROBE)

For open-weight models, reads internal activation values during processing and compares them to a pre-calibrated "safe" pattern, flagging cases closer to a "harmful request" pattern. Calibration is done separately per quantization level — the key idea that makes the system quantization-aware.

6.3 Black-Box Fallback Module (B-PROBE)

For closed models where internals aren't accessible, looks at the probabilities of the first few generated words and uses a small trained classifier to estimate whether the model is refusing or complying; also generates reworded versions of the same prompt and checks answer consistency.

6.4 Streaming Output Monitor (TRIPWIRE)

Tracks the model's "confidence"/"uncertainty" word-by-word during generation. Sudden unusual shifts can indicate an injected instruction has pushed the model off its normal behaviour. Implemented as a lightweight running calculation, requiring no full-response buffering.

6.5 Output Judge and Adaptive Controller (LOOKOUT + CONDUCTOR)

A lightweight judge (small classifier) reviews the generated output for signs that safety instructions were bypassed, and checks whether a hidden "canary" marker placed in system instructions leaked into the output (indicating a successful injection). Judge results feed a simple adaptive controller that learns over time which combination of checks works best and adjusts strategy accordingly.

6.6 Hardware Router (LADDER)

Detects the device tier at start-up and enables only the checks the device can comfortably run — e.g., 8 GB devices run the full check set, 2 GB devices run a reduced, faster set. A simple weight-verification step (ATTEST) also checks the model file's hash against a known-good value at start-up to detect tampering.

7. Methodology

The project proceeds in five phases:

- Phase 1 – Study and Setup: review existing defense tools and published findings on quantization/safety degradation; set up Python, PyTorch, Hugging Face Transformers, and bitsandbytes on Colab and a local machine.
- Phase 2 – Core Module Development: implement each of the six modules as separate, testable Python components, unit-tested on synthetic data before connecting to a real model.
- Phase 3 – Calibration and Integration: run Llama-3.2-3B-Instruct at FP16 and NF4 (later GGUF), calibrate detection thresholds per version using a benign prompt dataset, and connect all modules into one pipeline.
- Phase 4 – Edge Deployment: deploy the integrated system on a Raspberry Pi 5 running a quantized model via llama.cpp, and verify the hardware router correctly reduces active checks to match available memory.
- Phase 5 – Testing and Evaluation: test using a public benchmark of harmful/benign prompts (e.g., AdvBench), measuring blocked attacks, false positives on normal requests, and added latency per check; document results and prepare the final report.

8. Hardware and Software Requirements

8.1 Hardware

- Development laptop/desktop with internet access.
- Cloud GPU environment (Google Colab, free or paid tier) for calibration and testing.
- Raspberry Pi 5 (8 GB RAM) for edge deployment testing.
- Optional: lower-memory SBC (e.g., Raspberry Pi 4, 2–4 GB) for reduced-check configuration testing.

8.2 Software

- Python 3.10+; Hugging Face Transformers and bitsandbytes for loading/quantizing the model.
- llama.cpp for running GGUF-quantized models on the Raspberry Pi; PyTorch as the ML framework.
- Git/GitHub for version control; pytest for module testing.
- Public datasets: Anthropic-HH-RLHF (calibration), AdvBench (testing).

9. Budget

As a software-focused project, the budget is modest — mainly cloud compute time and one or two edge devices for real-world demonstration.

Item	Purpose	Est. Cost (NPR)
Cloud GPU credits (Colab Pro / rented A100)	Running/testing quantized models	6,000–10,000
Raspberry Pi 5 (8 GB)	Mid-tier edge hardware testing	15,000
Small SBC (Orange Pi / Pi 4, 2–4 GB)	Low-resource edge tier testing	8,000
MicroSD cards, cables, power adapters	Supporting hardware	3,000
Internet and cloud storage	Dataset and model downloads	2,000
Miscellaneous / contingency	Unplanned costs	3,000
Total (approximate)		**37,000–41,000**

Note: figures are approximate and may be reduced by using free-tier cloud resources (e.g., Colab's free GPU tier) wherever possible.

10. Timeline

Planned for completion within a 16-week semester:

Weeks	Activity	Deliverable
1–2	Literature study, environment setup, dataset collection	Project plan finalized, tools installed
3–4	Build VESTIBULE input gates and ATTEST hash checker	Working input-filtering module
5–7	Build PROBE white-box observer; calibrate on FP16/NF4 models	Calibrated refusal-margin detector
8–9	Build TRIPWIRE streaming monitor and B-PROBE black-box fallback	Real-time monitoring module
10–11	Build LOOKOUT output judge and CONDUCTOR gate selector	End-to-end defense pipeline
12	Build LADDER hardware router; deploy on Raspberry Pi	Working edge deployment
13–14	Testing against GGUF-quantized models and attack prompts	Test results and performance report
15–16	Bug fixing, documentation, final report and presentation	Final project submission

11. Expected Outcomes

- A working, modular CLIFFGUARD prototype with all six modules implemented and integrated.
- A calibrated detection system tested on a real open-source LLM at two or more quantization levels.
- A working demonstration on at least one real edge device (Raspberry Pi 5).
- A measured comparison of detection accuracy and response speed with and without CLIFFGUARD.
- A documented codebase and final report describing design, implementation, and test results, suitable for future extension.

12. Future Scope

- Extending calibration/testing to more heavily compressed formats (3-bit, 2-bit GGUF), where the safety cliff is expected to be most severe.
- Testing across additional model families (Qwen, Mistral) to check generalisation.
- Replacing the simplified adaptive controller with a fully trained online-learning policy once enough usage data is available.
- Packaging the system as an installable library/plugin for other edge-AI applications.
- Publishing findings and an anonymised test dataset for the research and developer community.

13. Conclusion

CLIFFGUARD addresses a practical, growing problem: as LLMs move from cloud servers onto small, everyday edge devices, their safety behaviour can quietly weaken from compression while they remain exposed to prompt injection. This project takes a hands-on engineering approach, building a layered, hardware-aware defense pipeline that can be calibrated, tested, and deployed on real, low-cost hardware within a single semester — a working prototype demonstrating a practical path towards safer edge-AI deployment, with a clear foundation for further development.

14. References

[1] Arditi, A., et al. "Refusal in Language Models Is Mediated by a Single Direction." 2024.

[2] Zhao, et al. "Separating Harmfulness and Refusal Directions in LLM Representations." NeurIPS, 2025.

[3] Egashira, K., et al. "Cliff-Exploit Attacks via Post-Training Quantization." NeurIPS, 2024.

[4] Hong, et al. "Mind the Gap: Quantization and Safety Alignment." ICLR, 2025.

[5] Chao, P., et al. "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models." 2024.

[6] Zou, A., et al. "AdvBench: Universal and Transferable Adversarial Attacks on Aligned Language Models." 2023.

[7] Meta AI. "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations." 2023.

[8] NVIDIA. "NeMo Guardrails Documentation." Accessed 2026.

[9] Hugging Face. "Transformers and bitsandbytes Documentation." Accessed 2026.

[10] llama.cpp Contributors. "llama.cpp: LLM Inference in C/C++." GitHub repository. Accessed 2026.